CS4051 — Information Retrieval

Week 04: Index Compression

Muhammad Rafi | February 16, 2026

---

1. Why Compression?

In Information Retrieval we deal with massive amounts of data — billions of documents, millions of terms. Storing all of this efficiently is critical.

Three Main Reasons

1. Save Disk Space — Less data = cheaper storage. At Google/Bing scale, saving even 1 byte per posting saves terabytes.

2. Keep More Data in RAM — RAM is ~100x faster than disk. If the index fits in memory, queries are answered in milliseconds.

3. Faster Disk-to-Memory Transfer — Even though decompression takes CPU time, reading compressed data and decompressing it is faster than reading the full uncompressed data, because disk I/O is the bottleneck.

> *Key Assumption: Decompression algorithms must be fast — and the ones used in IR are indeed very fast.*

Real-World Impact

Metric	Value
Index size vs. collection	Only 4% of total collection size
Index size vs. raw text	Only 10–15% of raw text size

---

2. Heap's Law

As you add more documents to a collection, your vocabulary (unique words) keeps growing. Heap's Law tells us **how fast** it grows.

Formula: M = k T^b

Symbol	Meaning	Typical Value
M	Vocabulary size (unique terms)	—
T	Total tokens in collection	—
k	Constant	30 – 100

Symbol	Meaning	Typical Value
b	Constant	0.4 – 0.6

For RCV1 (Reuters News Collection)

```
M = 10^1.64 x T^0.49 ~ 44 x T^0.49 (k ~ 44, b = 0.49)
```

Since b ≈ 0.5, vocabulary grows roughly as the **square root** of collection size. If the collection grows 100x bigger, the vocabulary grows only ~10x.

Effect of Processing Techniques

Technique	Effect on Vocabulary Growth
Case-folding (Apple → apple)	Slows growth
Stemming (running → run)	Slows growth
Including numbers	Speeds growth
Spelling errors	Speeds growth

Key Conclusions

1. Vocabulary **never stops growing** — new words, names, slang and typos keep appearing.
2. Vocabulary gets **very large** for large collections — which is exactly why dictionary compression is needed.

---

3. Zipf's Law

Heap's Law told us *how many* unique words exist. Zipf's Law tells us *how often* each word appears.

The Law

The **i-th most frequent term** has frequency proportional to 1/i:

```
cf_i = a / i
```

Symbol	Meaning
cf_i	Collection frequency of the i-th ranked term
i	Rank of the term (1 = most frequent)
a	Normalizing constant (numerically = frequency of rank-1 term)

Concrete Example

If "the" (rank 1) appears 61,847 times, then a = 61,847, and:

Rank	Word	Expected (a/i)	Actual
1	the	61,847	61,847
2	of	30,924	29,391
3	and	20,616	26,817
800	friends	77	~10

Zipf's Law is an approximation — actual values are close but not exact.

Log-Log Relationship

Taking log of both sides: log(cf_i) = log(a) - log(i)

This gives a straight line on a log-log graph — which is the empirical proof that Zipf's Law holds in real language.

Rank x Frequency ~ Constant (from slide data)

Rank	Word	Frequency	R x F
10	he	877	8,770
20	but	410	8,200
30	be	294	8,820
800	friends	10	8,000
1000	family	8	8,000

IR Consequences

Stop words are inevitable — a tiny number of words appear in almost every document and are useless for search.

Rare words are most useful — high-rank words are highly descriptive and discriminating.

Posting list sizes are very unequal — 'the' has millions of entries; 'arachnocentric' has maybe 3. This drives the need for variable-length compression.

---

4. Dictionary Compression

The dictionary maps terms to their postings and must fit in main memory for fast retrieval.

Naive Approach — Fixed Width Storage (20 bytes/term)

Field	Size
Term (fixed width)	20 bytes

Field	Size
Frequency	4 bytes
Pointer to Postings	4 bytes
Total	28 bytes / term

```
400,000 terms x 28 bytes = 11.2 MB
```

Why Fixed Width is Wasteful

Word	Actual Size	Bytes Wasted (out of 20)
"a"	1 byte	19 bytes
"be"	2 bytes	18 bytes
"cat"	3 bytes	17 bytes
"international"	13 bytes	7 bytes

Better Approach — String Compression

Store all terms concatenated in one long string, and keep a pointer in the dictionary table to where each term starts.

```
....systilesyzygeticsyzygialsyzygyszaibelyiteszczecinszomo....
```

The pointer to the *next* word implicitly shows where the current word ends — no need to store word length.

New Space Calculation

Field	Size	Reason
Frequency	4 bytes	Same as before
Pointer to Postings	4 bytes	Same as before
Pointer to Term String	3 bytes	log2(3.2M) = 22 bits ~ 3 bytes
Avg term in string	8 bytes	Average English dictionary word
Total	19 bytes / term	vs 28 bytes before

```
400,000 terms x 19 bytes = 7.6 MB (vs 11.2 MB) ~32% reduction
```

Why 3 bytes for term pointer? Total string = 400K x 8B = 3.2MB. log2(3,200,000) = 22 bits = 3 bytes.

---

5. Postings Compression

The postings file is **at least 10x larger** than the dictionary, so compressing it has an even bigger impact.

Naive Storage — 32 bits per docID

For RCV1 with 800,000 documents: log2(800,000) ≈ 20 bits needed, but we typically use 32 bits (4 bytes). Our goal: use far fewer than 20 bits per docID.

Key Insight — Store Gaps Instead of DocIDs

Instead of storing actual docIDs, store the **difference (gap)** between consecutive docIDs.

Term	Actual DocIDs	Gaps Stored
"the"	1, 2, 3, 4, 5, ...	1, 1, 1, 1, 1, ...
"arachnocentric"	283, 500, 802	283, 217, 302

Why Gaps Help

Term	Avg Gap	Bits Needed	vs Fixed 20 bits
"the"	~1	~1 bit	95% saving
"computer"	~100	~7 bits	65% saving
"arachnocentric"	~300	~9 bits	55% saving

Variable Length Encoding

Goal: if the average gap for a term is G, use approximately log2(G) bits per gap.

This requires codes that are **short for small numbers** and longer for large numbers — unlike fixed-width which wastes space on small gaps.

Gamma Encoding

Gamma encoding works in two parts:

Step 1 — Write the number in binary:

```
732 in binary = 1011011100 (10 digits)
```

Step 2 — Encode as:

- **Length part:** (number of binary digits - 1) zeros → 9 zeros = 000000000
- **Offset part:** full binary representation → 1011011100

```
Gamma(732) = 000000000 1011011100
```

Decoding Gamma Code

Given: 0000000001011011100

Step	Action	Result
1	Count leading zeros	9 zeros
2	Take next 9+1 = 10 bits	1011011100
3	Convert to decimal	732 ✓

Gamma Encoding — Size Comparison

Number	Binary	Gamma Code	Bits Used
1	1	1	1 bit
2	10	010	3 bits
3	11	011	3 bits
7	111	00111	5 bits
732	1011011100	000000000 1011011100	19 bits

Small numbers get very short codes — perfect for frequent terms with tiny gaps!

Summary

Topic	Key Formula / Technique	Key Takeaway
Why Compression	—	Speed + space; decompress faster than read raw
Heap's Law	M = k T^b	Vocabulary grows forever; never saturates
Zipf's Law	cf_i = a / i	Few very frequent, many very rare terms
Dictionary Compression	String storage + pointers	11.2 MB → 7.6 MB (~32% saving)
Postings Compression	Gap encoding + Gamma code	Variable bits; ~1 bit for 'the' gaps

A well-compressed IR index can be as small as **4% of the total collection size** and **10–15% of the raw text size**, while still supporting fast Boolean retrieval.